Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Лабораторна робота №4

Методологія інженерії програмного забезпечення

Тема: Formal Theory, Computer Simulations

Виконав:

студент групи ІП-51мн

Панченко С. В.

Перевірив:

Щебланін Ю. М.

Київ 2026

ЗМІСТ

1 Мета 3
2 Завдання 4
3 Виконання комп’ютерного моделювання 5
3.1 Аналіз Runtime-помилок згенерованого коду 5
3.1.1 Devstral: Втрата контексту об’єкта (Scope Error) 5
3.1.2 DeepSeek Coder: Проблема глобального стану (Global Scope Issue) 5
3.1.3 Qwen3 Coder: Конфлікт патернів (Context Manager Conflict) 6
3.2 Оновлені результати симуляції 6
4 Формальна теорія 9
4.1 Теорія архітектурної ілюзії та локального контексту LLM 9
5 Висновок 10
Перелік посилань 11
А Лістинг коду 12
А.1 deepseek.py 12
А.2 devstral.py 13
А.3 qwen3.py 16
А.4 Devstral (Помилка області видимості) 20
А.5 Qwen3 Coder (Конфлікт управління ресурсами) 21
А.6 DeepSeek Coder (Помилка глобального стану) 21
А.7 prompt.txt 21

1 МЕТА

Ознайомитися з типами досліджень, які виконуються в будь-яких умовах: формальними теоріями (Formal Theory) та комп'ютерним моделюванням (Computer Simulations) [1]. Дослідити ефективність застосування великих мовних моделей (LLM) [2] у повному циклі розробки програмного забезпечення та проаналізувати архітектурні вразливості згенерованого коду.

2 ЗАВДАННЯ

1. Визначити мету дослідження та обрати метод дослідження (Computer Simulations with Software Subjects).
2. Створити план дослідження та обрати інструменти (локальне середовище Ollama, моделі `qwen3-coder:30b`, `devstral:latest`, `deepseek-coder:33b`).
3. Виконати дослідження (симуляцію розробки застосунку за 6 аспектами).
4. Аналізувати дані дослідження (включаючи runtime-помилки згенерованого коду) та зробити висновки.
5. Сформулювати формальну теорію щодо спроможності LLM автономно вирішувати інженерні задачі.
6. Створити звіт за результатами дослідження.

3 ВИКОНАННЯ КОМП'ЮТЕРНОГО МОДЕЛЮВАННЯ

Для виконання експерименту було створено комп'ютерну симуляцію повного циклу розробки консольної гри "Змійка" (бібліотека `curses` мови Python). В якості об'єктів дослідження (Software Subjects) [3] виступили три локально розгорнуті мовні моделі з великою кількістю параметрів (30B+): Qwen3 Coder, Devstral та DeepSeek Coder.

Моделювання відбувалося у контрольованих умовах (однаковий системний промпт), що виключало вплив людського фактору на етапі генерації. Важливим аспектом симуляції стала перевірка згенерованого коду на працездатність (Execution Test) без попереднього втручання людини.

3.1 Аналіз Runtime-помилок згенерованого коду

Під час експерименту було виявлено критичний феномен: жодна з трьох моделей не змогла згенерувати повністю робочий код, незважаючи на успішну генерацію документації та тестів [4]. Детальний аналіз помилок виявив фундаментальні проблеми в розумінні моделями архітектури та області видимості змінних (Scope):

3.1.1 Devstral: Втрата контексту об'єкта (Scope Error)

Модель Devstral використала об'єктно-орієнтований підхід, проте в головному циклі гри спробувала звернутися до локальної змінної `key`, якої не існувало в поточній області видимості (була присутня лише властивість класу `self.key`). Це свідчить про те, що лінійна генерація токенів призвела до втрати логічного зв'язку між ініціалізацією атрибутів у `__init__` та їх використанням у методах класу.

3.1.2 DeepSeek Coder: Проблема глобального стану (Global Scope Issue)

Модель обрала процедурний підхід. Змінна `snake` була визначена локально всередині функції `main()`. Проте допоміжна функція `add_food()`, згенерована моделлю нижче по коду, намагалася отримати доступ до цієї змінної напряму. Модель не передала `snake` як аргумент функції і не задекларувала її як `global`. Це типова помилка, викликана тим, що LLM розглядає код як єдиний текстовий потік, часто ігноруючи жорсткі правила інкапсуляції в Python [4].

3.1.3 Qwen3 Coder: Конфлікт патернів (Context Manager Conflict)

Ця помилка найцікавіша з точки зору архітектури нейромереж. Промпт вимагав використання безпечної обгортки `curses.wrapper(main)` [5]. Ця функція в Python автоматично ініціалізує термінал, а при завершенні (або помилці) автоматично викликає `curses.endwin()` для відновлення стану терміналу. Проте Qwen3, “пам’ятаючи” тисячі прикладів ручного управління curses з навчальної вибірки, додатково додала виклик `curses.endwin()` всередині свого коду. Коли програма завершувалась, відбувався подвійний виклик звільнення ресурсів, що призводило до краху обгортки. Модель “змішала” два різних архітектурних патерни в один.

3.2 Оновлені результати симуляції

З урахуванням виявлених помилок, порівняльна таблиця (3.1) була скоригована. До часу розробки за допомогою LLM було додано необхідний час на ручний дебагінг та виправлення архітектурних недоліків розробником-людиною.

Таблиця 3.1 — Скоригований аналіз ефективності з урахуванням виявлених Runtime-помилок

Аспект	**Складова частина**	**Альтерн. спосіб (Ручний)**	**Випадок для експерименту (LLM)**	**Час (LLM + Дебаг)**	**Час (Ручний)**	**Рекомендації**
Вимоги	Функціональні та нефункціональні вимоги	Ручний аналіз специфікації	Генерація вимог на базі ТЗ	1 хв	20 хв	Ідеально для створення чернетки проєкту. Працює без помилок.
Кодування	Game Loop, логіка колізій `curses`	Написання коду розробником з нуля	Автоматична генерація скрипта	**15 хв** (через дебаг)	60 хв	Генерує Boiler-plate, але **критично потребує** ручного виправлення видимості змінних.
Тестування	Unit-тести логіки гри	Ручне написання тестів (Mock)	Генерація `unittest` кейсів	**10 хв**	30 хв	Згенеровані тести часто перевіряють неправильні сутності або ламаються через глобальні змінні.
Розгортання	Інструкції з запуску	Ручне складання `README`	Генерація інструкцій деплою	1 хв	10 хв	Ефективно вирішується LLM без втручання.

Аспект	Складова частина	Альтерн. спосіб (Ручний)	Випадок для експерименту (LLM)	Час (LLM + Дебаг)	Час (Ручний)	Рекомендації
Підтримка	Пошук потенційних багів	Читання документації бібліотеки	Запит на пошук вразливостей	2 хв	40 хв	Моделі теоретично знають баги (напр., SIGWINCH), але не можуть їх уникнути у власному коді.
Документація	Створення опису керування	Ручне написання тексту	Автогенерація Docstrings	1 хв	15 хв	Можна повністю довірити генерацію базової документації ШІ.

4 ФОРМАЛЬНА ТЕОРІЯ

На основі емпіричних даних, отриманих під час симуляції (зокрема, 100% відмови згенерованого коду при спробі виконання), розроблено наступну формальну теорію.

4.1 Теорія архітектурної ілюзії та локального контексту LLM

Комп'ютерне моделювання доводить, що великі мовні моделі (LLM) володіють високою здатністю до синтезу синтаксично коректного коду (Boiler-plate) та документації, скорочуючи час на ці етапи до 90%. Однак, при створенні систем, що залежать від станів (stateful applications), керування пам'яттю або складних життєвих циклів (наприклад, TUI інтерфейси на базі `curses`), моделі неминуче страждають на архітектурну ілюзію [6].

Це явище проявляється у нездатності моделі утримувати строгу ієрархію видимості змінних (Scope) між різними функціями та у схильності контамінувати (змішувати) взаємовиключні архітектурні патерни (наприклад, одночасне використання автоматичних обгорток та ручного звільнення ресурсів).

Висновок теорії: Автономна генерація складних програмних модулів за допомогою поточного покоління LLM є неможливою без етапу глибокого інженерного рев'ю. Ефективність ШІ є максимальною лише на етапах Solution Seeking (пошук рішень) [1], написання ізольованих чистих функцій та генерації артефактів природною мовою (документація, вимоги).

5 ВИСНОВОК

Під час виконання лабораторної роботи №4 було застосовано метод комп'ютерного моделювання (Computer Simulation) для оцінки реальної ефективності використання LLM у процесі розробки програмного забезпечення. Експеримент, проведений на трьох потужних локальних моделях (Qwen3-30B, Devstral, DeepSeek-33B), дав неочікувані, але цінні результати.

Жодна з моделей не змогла створити робочий прототип гри без синтаксичних або логічних помилок. Аналіз стеків виклику (Stack Trace) показав, що головною слабкістю ШІ є робота з контекстом (Global/Local Scope) та неправильне застосування патернів управління ресурсами (double-free помилки в `curses`).

Сформульована Теорія архітектурної ілюзії пояснює цей феномен тим, що LLM генерують код лінійно і схильні поєднувати фрагменти різних підходів з навчальної вибірки, ігноруючи загальну консистентність програми. Тим не менш, результати свідчать, що використання ШІ є вкрай ефективним для написання супровідної документації, формування вимог та створення базової структури класів, залишаючи за людиною-інженером роль архітектора та тестувальника.

ПЕРЕЛІК ПОСИЛАНЬ

[1] T. C. Lethbridge, S. E. Sim, and J. Singer, *Studying Software Engineers: Data Collection Techniques for Software Field Studies*, vol. 10. Springer, 2005, pp. 311–341.

[2] S. Bubeck *et al.*, "Sparks of artificial general intelligence: Early experiments with GPT-4," *arXiv preprint arXiv:2303.12712*, 2023.

[3] D. Russo and K.-J. Stol, "Soft theory: a pragmatic alternative to conduct quantitative empirical studies," in *Proceedings of the Joint 7th International Workshop on Conducting Empirical Studies in Industry*, 2019, pp. 30–33.

[4] J. Finnie-Ansley, P. McCallum, P. Denny, B. A. Becker, and A. Luxton-Reilly, "The robots are coming: Exploring the implications of OpenAI codex on introductory programming," in *Australasian Computing Education Conference*, 2022, pp. 10–19.

[5] Python Software Foundation, "Curses Programming with Python." 2024.

[6] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

Додаток А

Лістинг коду

А.1 deepseek.py

```
import curses
from random import randint

# Constants
WIDTH = 60
HEIGHT = 20
TIMEOUT = 100
SNAKE_X = WIDTH // 4
SNAKE_Y = HEIGHT // 2
TAIL_LEN = 3
FOOD = None
SCORE = 0
DIRECTION = curses.KEY_RIGHT

# Initialize the screen
screen = curses.initscr()
curses.curs_set(0)
screen.timeout(TIMEOUT)

def add_food():
    global FOOD
    while FOOD is None:
        new_food = [randint(1, WIDTH - 2), randint(1, HEIGHT - 2)]
        if new_food not in snake:
            FOOD = new_food

def draw():
    screen.clear()
    screen.addstr(0, 0, f'Score: {SCORE} ')
    for body in snake:
        screen.addch(body[1], body[0], curses.ACS_CKBOARD)
    screen.addch(FOOD[1], FOOD[0], curses.ACS_DIAMOND)

def main():
    global SNAKE_X, SNAKE_Y, DIRECTION, SCORE, FOOD
```

```
    snake = [[SNAKE_X - i, SNAKE_Y] for i in range(TAIL_LEN)]
    add_food()

    while True:
        key = screen.getch()
        if key != -1:
            DIRECTION = key

        new_head = [snake[0][0], snake[0][1]]
        if DIRECTION == curses.KEY_RIGHT:
            new_head[0] += 1
        elif DIRECTION == curses.KEY_LEFT:
            new_head[0] -= 1
        elif DIRECTION == curses.KEY_UP:
            new_head[1] -= 1
        elif DIRECTION == curses.KEY_DOWN:
            new_head[1] += 1

        snake.insert(0, new_head)
        if snake[0] == FOOD:
            global SCORE
            SCORE += 1
            add_food()
            draw()
        else:
            snake.pop()
        draw()

if __name__ == '__main__':
    try:
        main()
    finally:
        curses.endwin()
```

A.2 devstral.py

```
import curses
import random
```

```
class SnakeGame:
    def __init__(self, stdscr):
        self.stdscr = stdscr
        curses.curs_set(0)
        self.sh, self.sw = self.stdscr.getmaxyx()
        self.win = curses.newwin(self.sh, self.sw, 0, 0)
        self.win.keypad(1)
        self.win.timeout(100)

        self.snake_x = self.sw//4
        self.snake_y = self.sh//2
        self.snake = [
            [self.snake_y, self.snake_x],
            [self.snake_y, self.snake_x-1],
            [self.snake_y, self.snake_x-2]
        ]

        self.food = [self.sh//2, self.sw//2]
        self.win.addch(int(self.food[0]), int(self.food[1]),
curses.ACS_PI)

        self.key = curses.KEY_RIGHT

    def main_loop(self):
        while True:
            next_key = self.win.getch()
            self.key = key if next_key == -1 else next_key

            if self.key == ord('q'):
                break

            new_head = [self.snake[0][0], self.snake[0][1]]

            if self.key == curses.KEY_DOWN:
                new_head[0] += 1
            if self.key == curses.KEY_UP:
                new_head[0] -= 1
```

```
            if self.key == curses.KEY_LEFT:
                new_head[1] -= 1
            if self.key == curses.KEY_RIGHT:
                new_head[1] += 1

            self.snake.insert(0, new_head)

            if (
                self.snake[0][0] in [0, self.sh] or
                self.snake[0][1] in [0, self.sw] or
                self.snake[0] in self.snake[1:]
            ):
                curses.endwin()
                quit()

            if self.snake[0] == self.food:
                self.food = None
                while self.food is None:
                    nf = [
                        random.randint(1, self.sh-1),
                        random.randint(1, self.sw-1)
                    ]
                    self.food = nf if nf not in self.snake else None
                self.win.addch(self.food[0], self.food[1],
curses.ACS_PI)
            else:
                self.snake.pop()

            self.win.addch(self.snake[0][0], self.snake[0][1],
curses.ACS_CKBOARD)

def main(stdscr):
    game = SnakeGame(stdscr)
    game.main_loop()

if __name__ == "__main__":
    try:
        curses.wrapper(main)
```

```
    except KeyboardInterrupt:
        curses.endwin()
        print("\nGame interrupted and terminal state restored.")
```

A.3 qwen3.py

```
import curses
import random
import time
import sys

class SnakeGame:
    def __init__(self, stdscr):
        self.stdscr = stdscr
        self.height, self.width = stdscr.getmaxyx()
        self.snake = [(self.height//2, self.width//2)]
        self.direction = curses.KEY_RIGHT
        self.food = None
        self.score = 0
        self.game_over = False
        self.setup_game()

    def setup_game(self):
        """Initialize game state and place first food"""
        self.stdscr.nodelay(True)
        self.stdscr.timeout(100)  # 100ms timeout for getch()
        self.generate_food()

    def generate_food(self):
        """Generate food at random location not occupied by snake"""
        while True:
            self.food = (random.randint(1, self.height-2),
random.randint(1, self.width-2))
            if self.food not in self.snake:
                break

    def draw(self):
        """Draw the game state to the screen"""
        self.stdscr.clear()
```

```
        # Draw border
        for y in range(self.height):
            for x in range(self.width):
                if y == 0 or y == self.height-1 or x == 0 or x ==
self.width-1:
                    self.stdscr.addch(y, x, '#')

        # Draw snake
        for i, (y, x) in enumerate(self.snake):
            char = 'O' if i == 0 else 'o'  # Head is different
character
            self.stdscr.addch(y, x, char)

        # Draw food
        if self.food:
            self.stdscr.addch(self.food[0], self.food[1], 'O')

        # Draw score
        self.stdscr.addstr(0, 2, f"Score: {self.score}")

        # Draw game over message
        if self.game_over:
            game_over_msg = "GAME OVER! Press 'R' to restart or 'Q' to
quit"
            self.stdscr.addstr(self.height//2, (self.width-
len(game_over_msg))//2, game_over_msg)

        self.stdscr.refresh()

    def update(self):
        """Update game state"""
        if self.game_over:
            return

        # Calculate new head position
        head_y, head_x = self.snake[0]
        if self.direction == curses.KEY_UP:
```

```
            new_head = (head_y - 1, head_x)
        elif self.direction == curses.KEY_DOWN:
            new_head = (head_y + 1, head_x)
        elif self.direction == curses.KEY_LEFT:
            new_head = (head_y, head_x - 1)
        elif self.direction == curses.KEY_RIGHT:
            new_head = (head_y, head_x + 1)
        else:
            new_head = (head_y, head_x)

        # Check for collisions with walls
        if (new_head[0] <= 0 or new_head[0] >= self.height-1 or
            new_head[1] <= 0 or new_head[1] >= self.width-1):
            self.game_over = True
            return

        # Check for collisions with self
        if new_head in self.snake:
            self.game_over = True
            return

        # Add new head
        self.snake.insert(0, new_head)

        # Check if food is eaten
        if new_head == self.food:
            self.score += 10
            self.generate_food()
        else:
            # Remove tail if no food eaten
            self.snake.pop()

    def handle_input(self):
        """Handle user input"""
        key = self.stdscr.getch()

        # Handle quit
        if key in [ord('q'), ord('Q')]:
```

```
            return False

        # Handle restart
        if key in [ord('r'), ord('R')] and self.game_over:
            self.restart_game()
            return True

        # Handle direction changes
        if key == curses.KEY_UP and self.direction != curses.KEY_DOWN:
            self.direction = curses.KEY_UP
        elif key == curses.KEY_DOWN and self.direction !=
curses.KEY_UP:
            self.direction = curses.KEY_DOWN
        elif key == curses.KEY_LEFT and self.direction !=
curses.KEY_RIGHT:
            self.direction = curses.KEY_LEFT
        elif key == curses.KEY_RIGHT and self.direction !=
curses.KEY_LEFT:
            self.direction = curses.KEY_RIGHT

        return True

    def restart_game(self):
        """Restart the game"""
        self.snake = [(self.height//2, self.width//2)]
        self.direction = curses.KEY_RIGHT
        self.score = 0
        self.game_over = False
        self.generate_food()

def main(stdscr):
    """Main game loop"""
    try:
        # Initialize curses
        curses.curs_set(0)  # Hide cursor
        stdscr.clear()
        stdscr.refresh()
```

```
        # Create game instance
        game = SnakeGame(stdscr)

        # Main game loop
        while True:
            # Handle input
            if not game.handle_input():
                break

            # Update game state
            game.update()

            # Draw game
            game.draw()

            # Small delay to control game speed
            time.sleep(0.1)

    except KeyboardInterrupt:
        pass
    except Exception as e:
        # Print error and exit cleanly
        stdscr.clear()
        stdscr.addstr(0, 0, f"Error: {str(e)}")
        stdscr.refresh()
        time.sleep(2)
    finally:
        # Always restore terminal state
        curses.endwin()

if __name__ == "__main__":
    curses.wrapper(main)
```

A.4 Devstral (Помилка області видимості)

```
Traceback (most recent call last):
  File "devstral.py", line 75, in <module>
    curses.wrapper(main)
  File "devstral.py", line 29, in main_loop
```

```
    self.key = key if next_key == -1 else next_key
NameError: name 'key' is not defined. Did you mean: 'self.key'?
```

A.5 Qwen3 Coder (Конфлікт управління ресурсами)

```
Traceback (most recent call last):
  File "qwen3.py", line 171, in <module>
    curses.wrapper(main)
  File "/usr/lib/python3.14/curses/__init__.py", line 101, in wrapper
    endwin()
_curses.error: endwin() returned ERR
```

A.6 DeepSeek Coder (Помилка глобального стану)

```
Traceback (most recent call last):
  File "deepseek.py", line 67, in <module>
    main()
  File "deepseek.py", line 38, in main
    add_food()
  File "deepseek.py", line 24, in add_food
    if new_food not in snake:
NameError: name 'snake' is not defined
```

A.7 prompt.txt

```
You are a Senior Software Engineer. I need to simulate a full software
development cycle for a "Snake" game using the Python curses library.

Please generate a SINGLE, complete, and executable Python file. Do not
output any text outside the Python code block.

Inside this Python file, structure the content exactly as follows:

    At the very top, create a large multi-line string (docstring
""" ... """) that serves as the project report. Inside this docstring,
include:

        Requirements Engineering: 3 functional and 2 non-functional
requirements.

        Deployment: Instructions to run the game and required
dependencies.

        Codebase Support: Identify 2 potential bugs specific to curses
```

and how to avoid them.

Documentation: A mini README with controls.

Below the docstring, write the complete, executable Game Code using curses. Ensure the terminal state is restored safely if the user exits.

At the very bottom of the file, include a small section for Testing. Write 2 basic unittest test cases to verify the snake's movement or collision logic (you can mock the curses window or test pure logic classes).

Output ONLY the raw Python code (you can use one ```python block).